

Day 18 - Part-1 – Docker and Docker Compose for GenAI

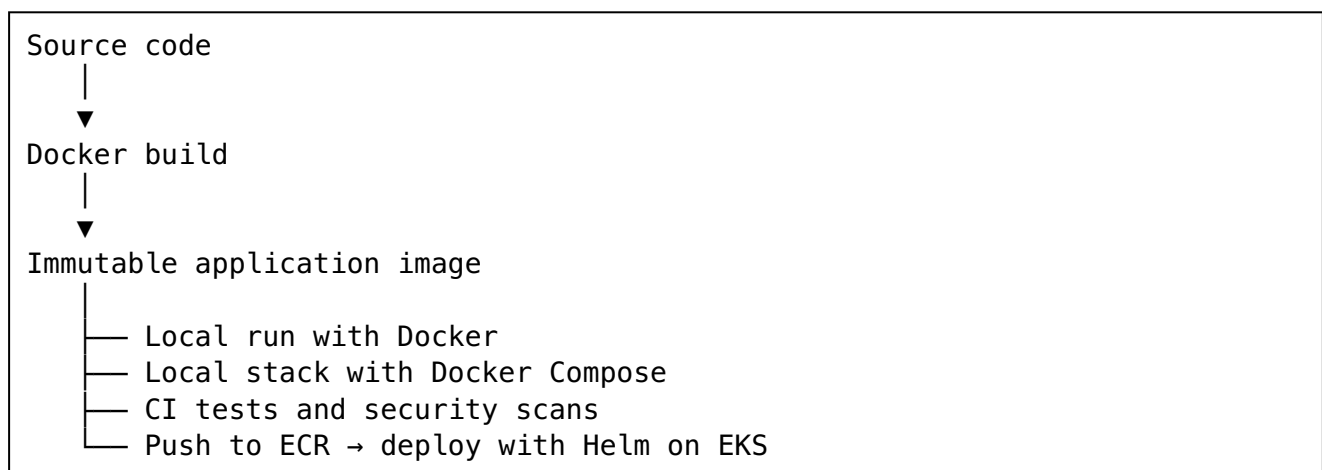
1. Learning objectives

By the end of this lesson, you should be able to:

- Explain Docker's image, container, layer, network, volume, registry, and runtime model.
- Build a secure FastAPI RAG image using a multi-stage Dockerfile.
- Run the same image as both an API and a background ingestion worker.
- Start PostgreSQL, Redis, Qdrant, the API, and worker through Docker Compose.
- Persist model caches and database data correctly.
- Debug networking, permissions, health checks, memory issues, architecture mismatches, and GPU visibility.
- Produce a Git-SHA-tagged, multi-platform image for your existing CI → ECR → Kubernetes/Helm flow.

2. Where Docker fits in your GenAI platform

For your RAG/agent platform, Docker provides a reproducible application package:



Docker handles application packaging. Docker Compose handles a multi-container application on one Docker host. Kubernetes handles scheduling and operating containers across a cluster.

The important principle is:

Build the application once, identify it using a Git SHA and digest, and promote that same artifact between environments.

Do not rebuild a different image for staging and production.

3. Container fundamentals

3.1 Image versus container

An **image** is a static, read-only package containing:

- Application code
- Runtime such as Python
- Installed dependencies

- Filesystem content
- Default command
- Image metadata

A **container** is a running instance of that image.

A useful analogy:

Class	→ Image
Object instance	→ Container
Executable file	→ Image
Running process	→ Container

A container remains running only while its main process remains running. When PID 1 exits, the container exits. (fastapi.tiangolo.com)

You can start multiple independent containers from the same image:

```
docker run --name api-1 genai-rag-api:1.0
docker run --name api-2 genai-rag-api:1.0
```

Each container starts with the same image content but receives its own writable container layer, process tree, network namespace, and runtime configuration.

3.2 Image layers and copy-on-write

A Docker image is composed of ordered filesystem layers.

Consider:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.lock .
RUN pip install -r requirements.lock
COPY app ./app
```

Conceptually, Docker creates layers such as:

```
Layer 1: Python base image
Layer 2: WORKDIR metadata
Layer 3: requirements.lock
Layer 4: installed dependencies
Layer 5: application code
```

Layers are normally read-only and reusable across images.

When a container changes a file, Docker writes the modified file into the container's writable layer instead of modifying the image. This is a **copy-on-write filesystem**.

Consequences:

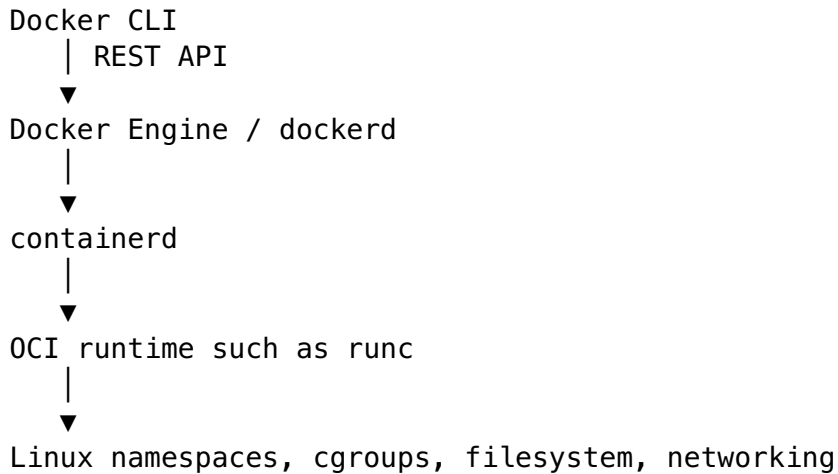
- Containers sharing an image can reuse the same underlying layers.
- Rebuilding only changed layers is faster.
- Deleting a file in a later layer does not necessarily remove its bytes from an earlier layer.

- Persistent data should not live in the writable container layer.
- Removing the container removes its writable layer.

Docker volumes persist independently from the container's writable layer. ([Docker Documentation](#))

3.3 Docker architecture

The main components are:



Docker client

The docker command:

```
docker build
docker run
docker ps
docker logs
```

Docker daemon

The Docker Engine daemon:

- Builds images
- Pulls and pushes images
- Creates networks and volumes
- Starts and stops containers
- Communicates with lower-level container runtimes

Docker Engine is a client-server technology containing the daemon, REST API, and CLI. ([Docker Documentation](#))

Registry

A registry stores:

- Image manifests
- Architecture-specific image manifests
- Layers
- Tags
- Digests
- Optionally SBOM and provenance attestations

Examples include Docker Hub and your already-covered AWS ECR registry.

Container runtime

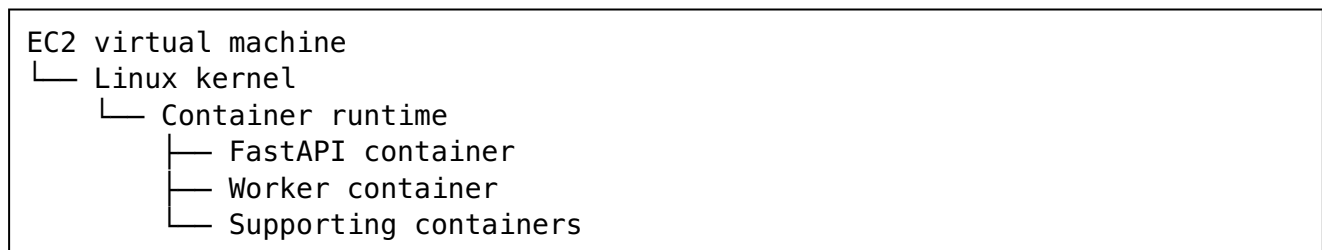
A low-level runtime such as runc creates the isolated Linux process according to the OCI runtime specification.

3.4 Container versus virtual machine

Area	Container	Virtual machine
Kernel	Shares host kernel	Has its own guest kernel
Isolation	Process-level isolation	Hardware/OS-level isolation
Startup	Usually seconds or less	Usually slower
Size	Often MB to several GB	Usually several GB or more
Density	High	Lower
OS flexibility	Must use compatible host kernel	Can run different guest OS
Security boundary	Generally weaker than a VM	Generally stronger boundary
Typical GenAI usage	API, worker, model server, vector DB	GPU node, build runner, cluster node

Containers share the host kernel while maintaining isolated processes, networking, and filesystems. This makes them lighter than full virtual machines. (fastapi.tiangolo.com)

In AWS, a common structure is:



3.5 Container lifecycle and exit codes

Typical states are:

created → running → paused/restarting → exited → removed
--

Useful exit codes:

Exit code	Common interpretation
0	Process completed successfully
1	General application failure
2	Invalid command or application usage
126	Command exists but cannot be executed
127	Command not found
137	Process received SIGKILL; often OOM or forced kill

143	Process received SIGTERM; often a normal container stop
-----	---

The formula for signal-based exits is commonly:

```
exit code = 128 + signal number
```

```
137 = 128 + SIGKILL(9)
```

```
143 = 128 + SIGTERM(15)
```

Inspect the result with:

```
docker inspect \
  --format='exit={{.State.ExitCode}} oom={{.State.OOMKilled}}
error={{.State.Error}}' \
  genai-api
```

4. Ports, environment variables, volumes, and networks

4.1 Port publishing

A container can listen on port 8000 internally without being reachable from the host.

Publish it using:

```
docker run -p 8000:8000 genai-rag-api:dev
```

Syntax:

```
-p HOST_PORT:CONTAINER_PORT
```

For safer local development, bind only to localhost:

```
docker run -p 127.0.0.1:8000:8000 genai-rag-api:dev
```

EXPOSE 8000 in a Dockerfile documents the expected port. It does not publish the port.

Inside Compose, containers communicate using the container port:

```
API → postgres:5432
API → redis:6379
API → qdrant:6333
```

They do not use published host ports for container-to-container traffic.

4.2 Environment variables

Pass runtime configuration using:

```
docker run \
  -e LOG_LEVEL=debug \
  -e QDRANT_URL=http://qdrant:6333 \
  genai-rag-api:dev
```

Or:

```
docker run --env-file .env genai-rag-api:dev
```

Dockerfile ENV supplies an image-level default:

```
ENV LOG_LEVEL=info
```

Runtime configuration can override it:

```
docker run -e LOG_LEVEL=debug genai-rag-api:dev
```

Do not treat environment variables as a secure secret store. Environment values can appear in container inspection, process information, diagnostics, or accidental logs. Docker recommends using secrets rather than environment variables for sensitive values. ([Docker Documentation](#))

4.3 Build arguments versus environment variables

ARG

Available during the image build:

```
ARG VCS_REF=unknown
LABEL org.opencontainers.image.revision="${VCS_REF}"
```

Build with:

```
docker build \
  --build-arg VCS_REF="$(git rev-parse HEAD)" \
  .
```

Use ARG for non-secret build metadata such as:

- Git SHA
- Build version
- Base-image selector
- Feature toggle that legitimately changes the artifact

Do not use it for passwords or API keys. Build arguments can appear in image metadata, history, cache, or provenance.

ENV

Becomes part of the image's runtime defaults:

```
ENV PYTHONUNBUFFERED=1
```

Use it for safe defaults, not credentials.

Build secrets

When a build genuinely needs a private package repository token, use BuildKit secrets:

```
RUN --mount=type=secret,id=pip_token \
  TOKEN="$(cat /run/secrets/pip_token)" && \
```

```
pip install --index-url "https://${TOKEN}@packages.example.com/simple"
my-package
```

Build it with:

```
docker buildx build \
  --secret id=pip_token,env=PRIVATE_PIP_TOKEN \
  .
```

BuildKit mounts the secret temporarily for that RUN instruction without baking it into the final image or normal layer cache. ([Docker Documentation](#))

4.4 Named volumes versus bind mounts

Named volume

Managed by Docker:

```
docker volume create hf_cache

docker run \
  --mount source=hf_cache,target=/home/app/.cache/huggingface \
  genai-rag-api:dev
```

Best for:

- PostgreSQL data
- Redis persistence
- Qdrant storage
- Hugging Face caches
- Docker-managed local state

Bind mount

Maps a specific host path:

```
docker run \
  --mount type=bind,source="$(pwd)/app",target=/app/app,readonly \
  genai-rag-api:dev
```

Best for:

- Live source-code editing
- Local configuration files
- Importing test datasets
- Debugging host-generated output

Bind mounts couple the container to the host path, permissions, and filesystem behavior. Named volumes are managed by Docker and are generally preferable for database and cache data. ([Docker Documentation](#))

4.5 Bridge networks and service-name DNS

Create a user-defined bridge network:

```
docker network create genai-network
```

Run services on it:

```
docker run -d \
  --name redis \
  --network genai-network \
  redis:7-alpine

docker run -d \
  --name api \
  --network genai-network \
  -e REDIS_URL=redis://redis:6379/0 \
  genai-rag-api:dev
```

Containers on the same user-defined network can resolve one another using container or service names. ([Docker Documentation](#))

A frequent mistake is:

```
REDIS_URL=redis://localhost:6379
```

Inside the API container, localhost means the API container itself.

Correct:

```
REDIS_URL=redis://redis:6379
```

4.6 Why application containers should remain stateless

Your FastAPI and ingestion-worker containers should not depend on their writable layers.

Store durable state in:

- PostgreSQL
- Qdrant
- S3 or another object store
- Redis when configured for appropriate persistence
- Explicit volumes for local development

Temporary files should go under /tmp or another bounded temporary filesystem.

A stateless application container can be:

- Replaced
- Scaled
- Restarted
- Rolled back
- Moved to another node

without losing application data.

5. Production-quality FastAPI Dockerfile

Assume this project structure:

```
.
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── worker.py
│   ├── config.py
│   └── services/
├── requirements.lock
├── Dockerfile
├── .dockerignore
├── docker-compose.yml
├── .env.example
└── secrets/
```

`requirements.lock` should contain exact dependency versions. For a deterministic build, generate a fully resolved lock with hashes.

Complete Dockerfile

```
# syntax=docker/dockerfile:1.7

# In CI, replace this tag with an approved digest, for example:
# python:3.12.11-slim-bookworm@sha256:<approved-digest>
ARG PYTHON_BASE=python:3.12.11-slim-bookworm

#
-----
# Stage 1: Build Python dependencies
#
-----

FROM ${PYTHON_BASE} AS builder

ENV PYTHONDONTWRITEBYTECODE=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

WORKDIR /build

# Build tools remain only in the builder stage.
RUN --mount=type=cache,target=/var/cache/apt,sharing=locked \
    --mount=type=cache,target=/var/lib/apt/lists,sharing=locked \
    apt-get update && \
    apt-get install -y --no-install-recommends \
        build-essential \
        libpq-dev

# Copy dependency metadata before source code so application changes do not
# invalidate the expensive dependency-installation layer.
```

```

COPY requirements.lock ./requirements.lock

RUN --mount=type=cache,target=/root/.cache/pip \
    python -m venv /opt/venv && \
    /opt/venv/bin/pip install \
        --require-hashes \
        --requirement requirements.lock

#
-----
# Stage 2: Small runtime image
#
-----
FROM ${PYTHON_BASE} AS runtime

ARG APP_UID=10001
ARG APP_GID=10001
ARG VCS_REF=unknown
ARG BUILD_DATE=unknown

LABEL org.opencontainers.image.title="genai-rag-api" \
    org.opencontainers.image.description="FastAPI RAG and agent backend" \
    org.opencontainers.image.revision="${VCS_REF}" \
    org.opencontainers.image.created="${BUILD_DATE}"

ENV PATH="/opt/venv/bin:${PATH}" \
    HOME="/home/appuser" \
    PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1 \
    HF_HOME="/home/appuser/.cache/huggingface" \
    HF_HUB_CACHE="/home/appuser/.cache/huggingface/hub" \
    PORT=8000

WORKDIR /app

# Install only runtime OS libraries.
RUN --mount=type=cache,target=/var/cache/apt,sharing=locked \
    --mount=type=cache,target=/var/lib/apt/lists,sharing=locked \
    apt-get update && \
    apt-get install -y --no-install-recommends \
        libgomp1 \
        libpq5 && \
    groupadd --gid "${APP_GID}" appgroup && \
    useradd \
        --uid "${APP_UID}" \
        --gid "${APP_GID}" \
        --create-home \
        --home-dir /home/appuser \
        --shell /usr/sbin/nologin \
        appuser && \
    mkdir -p \
        /home/appuser/.cache/huggingface \

```

```

    /tmp/app && \
chown -R appuser:appgroup \
    /home/appuser \
    /tmp/app

COPY --from=builder /opt/venv /opt/venv

# Source code changes invalidate only this late layer.
COPY --chown=appuser:appgroup ./app ./app

USER appuser:appgroup

EXPOSE 8000

# Liveness checks only the API process. Readiness should separately verify
# required dependencies such as PostgreSQL, Redis, and Qdrant.
HEALTHCHECK \
    --interval=30s \
    --timeout=3s \
    --start-period=20s \
    --retries=3 \
    CMD ["python", "-c", "import urllib.request;
urllib.request.urlopen('http://127.0.0.1:8000/health/live', timeout=2)"]

# Docker stop sends SIGTERM, allowing Uvicorn and FastAPI lifespan cleanup.
STOPSIGNAL SIGTERM

# Exec form makes Uvicorn the main process instead of placing a shell in
# front.
ENTRYPOINT ["python", "-m", "uvicorn"]

CMD [
    "app.main:app",
    "--host", "0.0.0.0",
    "--port", "8000",
    "--workers", "1",
    "--timeout-graceful-shutdown", "30",
    "--proxy-headers"
]
```

Current FastAPI guidance favors building from an official Python base, using exec-form commands, and normally letting an orchestrator manage container replication. The previously popular `tiangolo/uvicorn-gunicorn-fastapi` base image is deprecated. (fastapi.tiangolo.com)

5.1 Why this Dockerfile is production-oriented

Multi-stage build

The builder contains:

- Compiler
- Header files
- Dependency build tooling

- pip download cache

The runtime contains:

- Python runtime
- Installed virtual environment
- Required shared libraries
- Application source

This reduces runtime size and attack surface.

Dependency layer before source layer

This ordering:

```
COPY requirements.lock .  
RUN pip install ...  
COPY app ./app
```

means an application code change normally reuses the dependency layer.

Docker's official FastAPI guidance recommends copying dependency metadata before frequently changing application code to improve build cache reuse. (fastapi.tiangolo.com)

Non-root execution

The container runs as UID 10001, not root:

```
USER appuser:appgroup
```

Using a non-root runtime user is recommended by both Uvicorn and general container security guidance. ([Uvicorn](https://docs.uvicorn.org/en/latest/#running-as-non-root))

Correct signal handling

Exec form:

```
ENTRYPOINT ["python", "-m", "uvicorn"]
```

Correctly makes the Python process PID 1.

Avoid shell form:

```
CMD python -m uvicorn app.main:app
```

Shell form may introduce a shell process that interferes with signal delivery unless it correctly uses exec. FastAPI specifically recommends exec-form commands so graceful shutdown and lifespan events can run. (fastapi.tiangolo.com)

Health check

The image-level health check verifies process liveness:

```
GET /health/live
```

It does not check PostgreSQL, Redis, or Qdrant. Otherwise, a temporary dependency outage could classify a healthy application process as dead.

Graceful shutdown

Uvicorn supports a graceful-shutdown timeout, during which active responses and tasks can complete within configured limits. ([Uvicorn](#))

6. Complete .dockerignore

```
# Version control
.git
.git/**
.gitignore
.gitattributes

# CI and editor files
.github
.idea
.vscode
.DS_Store

# Secrets and local configuration
.env
.env.*
!.env.example
secrets/
*.pem
*.key
*.p12
*.crt

# Python caches and environments
__pycache__/
**/__pycache__/
*.py[cod]
*.pyo
.pytest_cache/
.mypy_cache/
.ruff_cache/
.coverage
coverage.xml
htmlcov/
.venv/
venv/

# Build output
build/
dist/
*.egg-info/
*.whl

# Local data and model artifacts
data/
datasets/
models/
```

```
model_weights/
hf_cache/
qdrant_storage/
postgres_data/
redis_data/

# Notebook state
.ipynb_checkpoints/
notebooks/output/

# Frontend dependencies, when using a monorepo
node_modules/
.next/
out/

# Logs and temporary files
*.log
logs/
tmp/
temp/
```

Without a `.dockerignore`, Docker may send unnecessary files—including `.env`, caches, local data, and credentials—to the build context. ([Docker Documentation](#))

Remember that `.dockerignore` does not replace `.gitignore`. Keep secrets excluded from both Git and the Docker build context.

7. FastAPI health and graceful-shutdown design

7.1 Liveness versus readiness

Liveness

Answers:

| Is the process running and able to serve a basic request?

```
@app.get("/health/live")
async def liveness() -> dict[str, str]:
    return {"status": "alive"}
```

Liveness should normally avoid remote dependencies.

Readiness

Answers:

| Can this instance currently serve normal application traffic?

A readiness check might verify:

- PostgreSQL connection
- Redis connection
- Qdrant availability

- Required configuration loaded
- Required embedding model loaded

```
@app.get("/health/ready")
async def readiness(request: Request) -> dict[str, str]:
    await request.app.state.postgres.execute("SELECT 1")
    await request.app.state.redis.ping()
    await request.app.state.qdrant.get_collections()
    return {"status": "ready"}
```

Use short timeouts. A health endpoint should not hang behind a failed dependency.

7.2 FastAPI lifespan cleanup

```
from contextlib import asynccontextmanager

from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup
    app.state.postgres = await create_postgres_pool()
    app.state.redis = await create_redis_client()
    app.state.qdrant = create_qdrant_client()

    try:
        yield
    finally:
        # Shutdown after SIGTERM
        await app.state.redis.aclose()
        await app.state.postgres.close()
        app.state.qdrant.close()

app = FastAPI(lifespan=lifespan)
```

FastAPI lifespan provides explicit startup and shutdown handling through the ASGI lifespan protocol. (fastapi.tiangolo.com)

Shutdown should:

- Stop accepting new work
- Complete or cancel bounded in-flight operations
- Flush logs and telemetry
- Close network clients
- Release database pools
- Stop background tasks

Do not begin an unbounded batch ingestion job during shutdown.

7.3 Worker signal handling

A custom ingestion worker must also handle SIGTERM.

```
import asyncio
import signal

async def run_worker() -> None:
    stop_requested = asyncio.Event()
    loop = asyncio.get_running_loop()

    for sig in (signal.SIGTERM, signal.SIGINT):
        loop.add_signal_handler(sig, stop_requested.set)

    while not stop_requested.is_set():
        job = await reserve_next_job(timeout_seconds=1)

        if job is None:
            continue

        try:
            await process_document(job)
            await acknowledge_job(job)
        except Exception:
            await retry_or_dead_letter(job)

    await close_worker_clients()

if __name__ == "__main__":
    asyncio.run(run_worker())
```

Important reliability rule:

| Acknowledge a queue job only after its durable side effects have completed.

That allows an interrupted ingestion job to be safely retried.

8. Uvicorn and Gunicorn considerations

Uvicorn directly

For your EKS-oriented image, the default should normally be:

```
uvicorn app.main:app --workers 1
```

Then scale containers at the orchestrator level.

Advantages:

- Straightforward signal handling
- One process has predictable memory
- Easier CPU and memory limits

- Cleaner logs and health checks
- Better alignment with Kubernetes replication

Multiple Uvicorn workers

On a single Docker Compose host, you could run:

```
uvicorn app.main:app --workers 4
```

But each worker is a separate process.

For GenAI applications, that may duplicate:

- Model weights in RAM
- Embedding model instances
- Connection pools
- In-memory caches
- GPU memory allocations

Four workers do not mean one shared model instance.

Gunicorn

Gunicorn was historically used to supervise Uvicorn workers. Modern Uvicorn can manage multiple workers directly, and current FastAPI guidance generally does not require the old prebuilt Gunicorn/FastAPI image. (fastapi.tiangolo.com)

Use Gunicorn only when your platform has a deliberate, tested Gunicorn standard. Do not add it automatically merely because an older tutorial uses it.

For a GPU-backed model process, multiple workers can duplicate VRAM and cause immediate out-of-memory failures. One model process per allocated GPU is usually a safer container boundary.

9. Complete docker-compose.yml

The following local development stack includes:

- FastAPI RAG/agent API
- Background ingestion worker
- PostgreSQL
- Redis
- Qdrant
- Persistent Hugging Face cache
- Optional model-cache warmup profile
- Health-based dependency startup
- Resource constraints
- Read-only application filesystems
- File-mounted secrets

Exact dependency image versions below are example pins. In controlled CI, use approved versions and preferably digest-pinned references.

```

name: genai-local

x-app-common: &app-common
build:
  context: .
  dockerfile: Dockerfile
  target: runtime
  args:
    VCS_REF: ${GIT_SHA:-local}
    BUILD_DATE: ${BUILD_DATE:-local}
image: genai-rag-api:${IMAGE_TAG:-dev}

env_file:
  - .env

environment:
  APP_ENV: local
  LOG_LEVEL: ${LOG_LEVEL:-info}

  DATABASE_HOST: postgres
  DATABASE_PORT: "5432"
  DATABASE_NAME: genai
  DATABASE_USER: genai
  DATABASE_PASSWORD_FILE: /run/secrets/postgres_password

  REDIS_URL: redis://redis:6379/0
  QDRANT_URL: http://qdrant:6333

  LLM_API_KEY_FILE: /run/secrets/llm_api_key

  HF_HOME: /home/appuser/.cache/huggingface
  HF_HUB_CACHE: /home/appuser/.cache/huggingface/hub
  EMBEDDING_MODEL_ID: ${EMBEDDING_MODEL_ID:-sentence-transformers/all-
MiniLM-L6-v2}
  EMBEDDING_MODEL_REVISION: ${EMBEDDING_MODEL_REVISION:-main}

secrets:
  - postgres_password
  - llm_api_key

networks:
  - backend

depends_on:
  postgres:
    condition: service_healthy
  redis:
    condition: service_healthy
  qdrant-health:
    condition: service_healthy

init: true
restart: unless-stopped

```

```

read_only: true

tmpfs:
  - /tmp:size=256m,mode=1777

security_opt:
  - no-new-privileges:true

cap_drop:
  - ALL

pids_limit: 256

logging:
  driver: json-file
  options:
    max-size: "10m"
    max-file: "3"

services:
  api:
    <<: *app-common

    command:
      - app.main:app
      - --host
      - 0.0.0.0
      - --port
      - "8000"
      - --reload
      - --timeout-graceful-shutdown
      - "30"

    ports:
      - "127.0.0.1:${API_PORT:-8000}:8000"

    volumes:
      # Development source-code bind mount
      - ./app:/app/app:ro

      # Model downloads persist across container rebuilds
      - hf_cache:/home/appuser/.cache/huggingface

  healthcheck:
    test:
      - CMD
      - python
      - -c
      - >
        import urllib.request;
        urllib.request.urlopen(
          'http://127.0.0.1:8000/health/ready',
          timeout=2

```

```

    )
    interval: 15s
    timeout: 3s
    start_period: 30s
    retries: 5

    stop_grace_period: 40s
    mem_limit: 2g
    cpus: 2.0
    shm_size: "512m"

worker:
  <<: *app-common

  # Override the Dockerfile's Uvicorn ENTRYPOINT.
  entrypoint:
    - python
    - -m
    - app.worker

  volumes:
    - ./app:/app/app:ro
    - hf_cache:/home/appuser/.cache/huggingface

  # The image's default health check expects an HTTP API.
  # A queue worker should instead expose a heartbeat/metric through
  # the queue or observability system.
  healthcheck:
    disable: true

  stop_grace_period: 60s
  mem_limit: 4g
  cpus: 3.0
  shm_size: "1gb"

postgres:
  image: postgres:16.9-bookworm

  environment:
    POSTGRES_DB: genai
    POSTGRES_USER: genai
    POSTGRES_PASSWORD_FILE: /run/secrets/postgres_password

  secrets:
    - postgres_password

  volumes:
    - postgres_data:/var/lib/postgresql/data

  ports:
    - "127.0.0.1:${POSTGRES_PORT:-5432}:5432"

```

```
networks:
  - backend

healthcheck:
  test:
    - CMD-SHELL
    - pg_isready -U genai -d genai
  interval: 10s
  timeout: 5s
  retries: 10
  start_period: 10s
```

```
restart: unless-stopped
mem_limit: 1g
cpus: 1.0
```

```
logging:
  driver: json-file
  options:
    max-size: "10m"
    max-file: "3"
```

```
redis:
  image: redis:7.4.5-alpine
```

```
command:
  - redis-server
  - --appendonly
  - "yes"
  - --maxmemory
  - 512mb
  - --maxmemory-policy
  - allkeys-lru
```

```
volumes:
  - redis_data:/data
```

```
ports:
  - "127.0.0.1:${REDIS_PORT:-6379}:6379"
```

```
networks:
  - backend
```

```
healthcheck:
  test:
    - CMD
    - redis-cli
    - ping
  interval: 10s
  timeout: 3s
  retries: 10
```

```
restart: unless-stopped
```

```
mem_limit: 768m
cpus: 0.75
```

```
logging:
  driver: json-file
  options:
    max-size: "10m"
    max-file: "3"
```

```
qdrant:
  image: qdrant/qdrant:v1.15.3
```

```
volumes:
  - qdrant_data:/qdrant/storage
```

```
ports:
  - "127.0.0.1:${QDRANT_PORT:-6333}:6333"
```

```
networks:
  - backend
```

```
restart: unless-stopped
mem_limit: 4g
cpus: 2.0
```

```
logging:
  driver: json-file
  options:
    max-size: "10m"
    max-file: "3"
```

```
# Qdrant's minimal image may not contain curl/wget.
# This tiny sidecar checks Qdrant's official readiness endpoint.
```

```
qdrant-health:
  image: curlimages/curl:8.12.1
```

```
command:
  - sh
  - -c
  - while true; do sleep 3600; done
```

```
networks:
  - backend
```

```
depends_on:
  qdrant:
    condition: service_started
```

```
healthcheck:
  test:
    - CMD
    - curl
```

```

    - --fail
    - --silent
    - --show-error
    - http://qdrant:6333/readyz
interval: 10s
timeout: 3s
retries: 20
start_period: 10s

restart: unless-stopped
mem_limit: 64m
cpus: 0.10

# Optional profile: pre-download a pinned embedding model into hf_cache.
# Start with:
# docker compose --profile models run --rm model-warmup
model-warmup:
  profiles:
    - models

  build:
    context: .
    dockerfile: Dockerfile
    target: runtime

  image: genai-rag-api:${IMAGE_TAG:-dev}

  env_file:
    - .env

  environment:
    HF_HOME: /home/appuser/.cache/huggingface
    HF_HUB_CACHE: /home/appuser/.cache/huggingface/hub
    EMBEDDING_MODEL_ID: ${EMBEDDING_MODEL_ID:-sentence-transformers/all-
MiniLM-L6-v2}
    EMBEDDING_MODEL_REVISION: ${EMBEDDING_MODEL_REVISION:-main}

  entrypoint:
    - python
    - -c

  command:
    - |
      import os
      from huggingface_hub import snapshot_download

      snapshot_download(
        repo_id=os.environ["EMBEDDING_MODEL_ID"],
        revision=os.environ["EMBEDDING_MODEL_REVISION"],
      )

  volumes:
    - hf_cache:/home/appuser/.cache/huggingface

```

```

networks:
  - backend

read_only: true

tmpfs:
  - /tmp:size=256m,mode=1777

security_opt:
  - no-new-privileges:true

cap_drop:
  - ALL

healthcheck:
  disable: true

restart: "no"

networks:
  backend:
    driver: bridge

volumes:
  postgres_data:
  redis_data:
  qdrant_data:
  hf_cache:

secrets:
  postgres_password:
    file: ./secrets/postgres_password.txt

  llm_api_key:
    file: ./secrets/llm_api_key.txt

```

Compose can wait for dependencies marked `service_healthy`; ordinary start ordering alone does not guarantee that a database is ready to accept connections. ([Docker Documentation](#))

Qdrant exposes `/healthz`, `/livez`, and `/readyz` monitoring endpoints. Its official container is intentionally minimal, so the example uses a small curl sidecar rather than modifying the database image just to install a health-check client. (api.qdrant.tech)

9.1 Local .env.example

```

IMAGE_TAG=dev
GIT_SHA=local
BUILD_DATE=local

```



```
API_PORT=8000
POSTGRES_PORT=5432
REDIS_PORT=6379
QDRANT_PORT=6333

LOG_LEVEL=info

EMBEDDING_MODEL_ID=sentence-transformers/all-MiniLM-L6-v2

# Pin an approved model commit SHA rather than using "main".
EMBEDDING_MODEL_REVISION=replace-with-model-commit-sha
```

Create your local file:

```
cp .env.example .env
```

Docker Compose supports both `.env` interpolation and `env_file` for passing runtime variables. Profiles allow optional services to remain disabled until explicitly selected. ([Docker Documentation](#))

9.2 Create local secret files

```
mkdir -p secrets

printf '%s\n' 'local-postgres-password' \
  > secrets/postgres_password.txt

printf '%s\n' 'replace-with-local-api-key' \
  > secrets/llm_api_key.txt

chmod 600 secrets/*.txt
```

Never commit this directory.

Compose makes explicitly granted secrets available as files inside `/run/secrets`. ([Docker Documentation](#))

A small configuration helper can support the `_FILE` pattern:

```
import os
from pathlib import Path

def read_config_or_secret(name: str) -> str | None:
    file_path = os.getenv(f"{name}_FILE")

    if file_path:
        return Path(file_path).read_text(encoding="utf-8").strip()

    return os.getenv(name)
```

Usage:

```
database_password = read_config_or_secret("DATABASE_PASSWORD")
llm_api_key = read_config_or_secret("LLM_API_KEY")
```

10. Docker Compose concepts

Services

Each service represents an independently replaceable container definition:

```
services:
  api:
  worker:
  postgres:
  redis:
  qdrant:
```

The API and worker use the same image but different commands.

Networks

All backend services join:

```
networks:
  backend:
```

This gives them service-name DNS:

```
postgres
redis
qdrant
```

Volumes

Named volumes preserve state:

```
volumes:
  postgres_data:
  redis_data:
  qdrant_data:
  hf_cache:
```

Secrets

Secrets are explicitly granted to selected services:

```
secrets:
  - postgres_password
  - llm_api_key
```

Profiles

The model warmup utility does not run during normal startup:

```
profiles:
  - models
```

Run it explicitly:

```
docker compose --profile models run --rm model-warmup
```

Development versus production-like Compose

The local API uses:

```
volumes:
  - ./app:/app/app:ro

command:
  - --reload
```

A production-like Compose configuration should:

- Remove source bind mounts
- Remove `--reload`
- Use an immutable registry image
- Use digest-pinned dependencies
- Obtain secrets from the deployment platform
- Avoid publishing database ports
- Use external log and metrics collection
- Use stricter network exposure

Docker recommends applying an additional Compose file over the base configuration when creating a production-specific configuration. ([Docker Documentation](#))

11. Starting the complete local GenAI stack

Validate the resolved configuration

Always inspect interpolation and merges first:

```
docker compose config
```

This catches:

- Missing variables
- Invalid YAML
- Missing secret files
- Incorrect anchors
- Invalid service references

Build and start

```
docker compose up --build -d
```

`docker compose up` builds when requested, creates required networks and volumes, starts services, and can aggregate logs when not detached. ([Docker Documentation](#))

Check service status

```
docker compose ps
```

Include exited utility containers:

```
docker compose ps --all
```

Follow API and worker logs

```
docker compose logs --follow --tail=200 api worker
```

Check API

```
curl http://localhost:8000/health/live  
curl http://localhost:8000/health/ready
```

Inspect Qdrant

```
curl http://localhost:6333/readyz
```

Warm the embedding-model cache

```
docker compose --profile models run --rm model-warmup
```

Stop without deleting containers

```
docker compose stop
```

Stop and remove containers and network

```
docker compose down
```

Also delete persistent data

```
docker compose down --volumes
```

Be careful: the last command deletes local PostgreSQL, Redis, Qdrant, and Hugging Face cache volumes.

12. Essential Docker commands

12.1 Build

```
docker build \  
  --pull \  
  --progress=plain \  
  .
```

```
--build-arg VCS_REF="$(git rev-parse HEAD)" \
--build-arg BUILD_DATE="$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
--tag genai-rag-api:dev \
.
```

`--pull` asks Docker to check for a newer version of the selected base reference. A digest pin remains necessary when you require an exact base artifact. Docker recommends minimal trusted bases and digest pins for stronger supply-chain integrity. ([Docker Documentation](#))

12.2 List images

```
docker image ls
```

12.3 Inspect image layers

```
docker history genai-rag-api:dev
```

12.4 Run a container

```
docker run --detach \
  --name genai-api \
  --publish 127.0.0.1:8000:8000 \
  --env-file .env \
  --read-only \
  --tmpfs /tmp:rw,noexec,nosuid,size=256m \
  --memory 2g \
  --cpus 2 \
  genai-rag-api:dev
```

This standalone command still needs reachable PostgreSQL, Redis, and Qdrant endpoints for readiness.

12.5 List running containers

```
docker ps
```

Include exited containers:

```
docker ps --all
```

12.6 Read logs

```
docker logs --follow --tail 200 genai-api
```

Since containers are disposable, applications should normally log to stdout and stderr rather than only writing internal log files.

12.7 Execute a debugging command

```
docker exec -it genai-api sh
```

Check identity:

```
docker exec genai-api id
```

Check environment safely:

```
docker exec genai-api \
  python -c "import os; print(os.getenv('QDRANT_URL'))"
```

Avoid dumping every environment variable because that may expose credentials.

12.8 Inspect configuration

```
docker inspect genai-api
```

Focused inspection:

```
docker inspect \
  --format='
  user={{.Config.User}}
  status={{.State.Status}}
  health={{if .State.Health}}{{.State.Health.Status}}{{else}}none{{end}}
  exit={{.State.ExitCode}}
  oom={{.State.OOMKilled}}
  image={{.Image}}
  ' \
  genai-api
```

12.9 Resource usage

```
docker stats
```

12.10 Stop gracefully

```
docker stop --time 40 genai-api
```

Docker waits for the configured timeout after sending the stop signal and then uses SIGKILL if the process has not exited. ([Docker Documentation](#))

12.11 Remove container

```
docker rm genai-api
```

Force removal:

```
docker rm --force genai-api
```

12.12 Tag image

```
docker tag \
  genai-rag-api:dev \
  registry.example.com/genai-rag-api:abc123def456
```

12.13 Push image

```
docker push registry.example.com/genai-rag-api:abc123def456
```

13. ENTRYPOINT versus CMD

A useful pattern is:

```
ENTRYPOINT ["python", "-m", "uvicorn"]  
CMD ["app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

The final process is:

```
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Running:

```
docker run genai-rag-api:dev \  
  app.main:app --host 0.0.0.0 --port 9000
```

replaces CMD but retains ENTRYPOINT.

To run the worker, Compose overrides ENTRYPOINT:

```
entrypoint:  
  - python  
  - -m  
  - app.worker
```

Exec form

```
CMD ["python", "-m", "app.main"]
```

No shell is inserted.

Shell form

```
CMD python -m app.main
```

Docker effectively runs:

```
/bin/sh -c "python -m app.main"
```

The shell may become PID 1 and complicate signal handling.

14. GenAI-specific container concerns

14.1 Keep model weights outside the application image

Avoid this:

```
COPY models/large-model /models/large-model
```

Problems:

- Very large image layers
- Slow upload and pull
- Slow autoscaling and cold starts
- Every model change forces an application image rebuild
- Application and model versions become tightly coupled
- Potential model-license redistribution concerns
- Large build contexts
- More registry storage and scanning overhead

Prefer:

```
Application image
├── Python runtime
├── inference/RAG code
└── model client libraries

Model artifact
├── object storage
├── model registry
├── node cache
├── named volume for local development
└── dedicated model-serving image only when intentionally designed
```

Track model version separately:

```
APPLICATION_IMAGE_DIGEST=sha256:...
EMBEDDING_MODEL_ID=...
EMBEDDING_MODEL_REVISION=<commit-sha>
PROMPT_VERSION=...
INDEX_VERSION=...
```

14.2 Safely download or mount models

A safe model-loading process should:

1. Use an approved repository.
2. Pin the model revision to a commit SHA.
3. Validate expected files and checksums where possible.
4. Review the model licence.
5. Avoid `trust_remote_code=True` unless reviewed and approved.
6. Download with limited credentials.
7. Write only to a designated cache directory.
8. Optionally switch the model directory to read-only after warmup.
9. Record the model revision in telemetry.

Avoid:

```
snapshot_download(
    repo_id="some-model",
```



```
    revision="main",  
)
```

Prefer:

```
snapshot_download(  
    repo_id="approved-org/approved-model",  
    revision="7f3c...exact-commit...",  
)
```

14.3 Hugging Face cache volume

The Compose example mounts:

```
- hf_cache:/home/appuser/.cache/huggingface
```

Benefits:

- Container rebuilds do not redownload the model.
- API and worker can reuse the local cache.
- Model warmup can happen before the normal services start.
- The application image remains small.

Permissions matter. The cache directory must be writable by UID 10001.

For production, be careful about sharing one writable filesystem between many simultaneous model processes. Test locking, filesystem semantics, throughput, and corruption recovery.

14.4 Large images and cold starts

Cold start includes more than process startup:

```
Image pull  
+ layer extraction  
+ container initialization  
+ dependency import  
+ model download  
+ model deserialization  
+ movement into RAM/VRAM  
+ readiness checks
```

Optimization techniques:

- Keep weights out of the general application image.
- Use minimal runtime images.
- Put stable dependencies in cacheable layers.
- Pre-pull required images on execution nodes where appropriate.
- Warm model caches.
- Avoid installing compilers in runtime images.
- Do not duplicate model weights across unnecessary workers.
- Size disk for both compressed layers and extracted model files.
- Use realistic readiness start_period values.

14.5 CPU versus GPU images

CPU container

Usually contains:

- CPU-specific PyTorch, ONNX Runtime, or inference library
- No CUDA libraries
- Smaller image
- Runs on ordinary nodes

GPU container

Usually contains:

- CUDA-compatible user-space libraries
- GPU-enabled framework build
- NVIDIA runtime integration
- Much larger runtime layers

The host still provides the NVIDIA kernel driver. The container contains compatible user-space CUDA libraries.

The NVIDIA Container Toolkit connects the Docker runtime to NVIDIA GPU devices. GPUs can be exposed using `--gpus`, while `NVIDIA_VISIBLE_DEVICES` controls device visibility. ([NVIDIA Docs](#))

Test GPU access:

```
docker run --rm \
  --gpus all \
  nvidia/cuda:12.9.0-base-ubuntu22.04 \
  nvidia-smi
```

NVIDIA documents this type of sample workload as the validation step after installing the host driver and Container Toolkit. ([NVIDIA Docs](#))

14.6 Compose GPU reservation

An optional GPU service can use:

```
services:
  local-model-server:
    image: your-approved-gpu-image:git-sha

    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities:
                - gpu
```

The host must have:

- Supported NVIDIA GPU
- Compatible host driver
- NVIDIA Container Toolkit
- Docker daemon configured for GPU access

For Compose GPU reservations, capabilities: [gpu] is required, and count and device_ids must not be used together. ([Docker Documentation](#))

14.7 Memory, CPU, shared memory, and disk

Memory

Docker containers have no memory limit unless you configure one. Without limits, a container can consume as much memory as the host allows. ([Docker Documentation](#))

Plan memory for:

```
Python runtime
+ model weights
+ runtime tensors
+ token/embedding buffers
+ vector batches
+ database clients
+ request concurrency
+ worker processes
+ filesystem cache
```

A 1 GB model file does not guarantee only 1 GB of RAM usage.

CPU

CPU-intensive embedding or reranking competes with:

- FastAPI event loop
- JSON processing
- Decompression
- Vector transformations
- Document parsing

Separate the API and ingestion worker so ingestion cannot consume all API CPU.

Shared memory

Docker's default /dev/shm can be too small for:

- PyTorch multiprocessing
- DataLoader workers
- Shared tensors
- Some browser/document-processing workloads

Compose:

```
shm_size: "1gb"
```

Do not increase it blindly; measure the workload.

Disk

Plan for:

- Image layers
- Docker build cache
- Extracted model files
- Hugging Face cache
- PostgreSQL WAL/data
- Qdrant segments and snapshots
- Redis AOF
- Temporary document extraction
- Logs

A full disk can make databases unhealthy even when CPU and memory look normal.

15. BuildKit and buildx

BuildKit is Docker's modern image-build backend. Buildx exposes BuildKit functionality including:

- Multi-platform builds
- Cache import/export
- Multiple builder instances
- Registry cache
- SBOM attestations
- Provenance attestations

Buildx extends the Docker CLI with BuildKit features. ([GitHub](#))

15.1 Create a builder

```
docker buildx create \  
  --name genai-builder \  
  --driver docker-container \  
  --use
```

Initialize it:

```
docker buildx inspect --bootstrap
```

The docker-container driver supports multi-architecture builds and advanced cache import/export. ([Docker Documentation](#))

15.2 Single-platform local build

```
docker buildx build \  
  --platform linux/amd64 \  
  --tag genai-rag-api:dev \  
  .
```

```
--load \  
.
```

--load imports the result into the local Docker image store.

15.3 Multi-platform build

```
GIT_SHA="$(git rev-parse --short=12 HEAD)"  
IMAGE="registry.example.com/genai-rag-api:${GIT_SHA}"  
  
docker buildx build \  
  --platform linux/amd64,linux/arm64 \  
  --build-arg VCS_REF="$(git rev-parse HEAD)" \  
  --build-arg BUILD_DATE="$(date -u +%Y-%m-%dT%H:%M:%SZ)" \  
  --tag "${IMAGE}" \  
  --push \  
.
```

A multi-platform tag points to a manifest list containing architecture-specific images. Buildx supports multi-platform builds for targets such as linux/amd64 and linux/arm64. ([Docker Documentation](#))

For a multi-platform result, use --push rather than trying to load all platforms into a classic local image store.

15.4 Remote registry cache

CI runners are often ephemeral, making external build cache especially useful. ([Docker Documentation](#))

```
GIT_SHA="$(git rev-parse --short=12 HEAD)"  
REPOSITORY="registry.example.com/genai-rag-api"  
IMAGE="${REPOSITORY}:${GIT_SHA}"  
CACHE="${REPOSITORY}:buildcache"  
  
docker buildx build \  
  --platform linux/amd64,linux/arm64 \  
  --cache-from type=registry,ref="${CACHE}" \  
  --cache-to type=registry,ref="${CACHE}",mode=max \  
  --build-arg VCS_REF="$(git rev-parse HEAD)" \  
  --build-arg BUILD_DATE="$(date -u +%Y-%m-%dT%H:%M:%SZ)" \  
  --tag "${IMAGE}" \  
  --push \  
.
```

Never place secrets into cacheable build arguments.

15.5 Generate SBOM and provenance

```
docker buildx build \  
  --platform linux/amd64,linux/arm64 \  
  --sbom=true \  
  --provenance=mode=max
```

```
--tag "${IMAGE}" \  
--push \  
.
```

An SBOM describes software components contained in or used to create the image. Provenance describes how the image was built. BuildKit can attach both to the resulting image as attestations. ([Docker Documentation](#))

16. Immutable image tagging

Git SHA tag

```
GIT_SHA="$(git rev-parse --short=12 HEAD)"  
IMAGE="registry.example.com/genai-rag-api:${GIT_SHA}"
```

Examples:

```
genai-rag-api:4a91c64f2b3e  
genai-rag-api:release-2026-07-30
```

A Git SHA tag is traceable but technically a registry tag can be moved unless registry policy prevents it.

Digest

After pushing:

```
docker buildx imagetools inspect "${IMAGE}"
```

A digest looks like:

```
sha256:4196c15...
```

The strongest deployment reference is:

```
registry.example.com/genai-rag-api@sha256:4196c15...
```

A digest identifies exact image content; tags are human-friendly pointers.

Build once, promote the same artifact

Correct:

```
Build image once  
→ push Git SHA tag  
→ test image  
→ promote same digest to staging  
→ promote same digest to production
```

Incorrect:

```
Build dev image
→ rebuild staging image
→ rebuild production image
```

Rebuilding can introduce a different:

- Base image
- Dependency resolution
- Package mirror result
- Timestamp
- Toolchain
- Architecture output

17. Handoff to your existing CI/ECR flow

The Docker-specific CI contract should be:

1. Resolve the Git SHA.
2. Build the image once with BuildKit.
3. Run tests and scans against that artifact.
4. Generate SBOM and provenance.
5. Push the Git-SHA tag.
6. Record the resulting digest.
7. Pass that tag or digest to the existing deployment pipeline.
8. Promote the same digest through environments.

ECR stores the image artifact. Helm should consume the immutable tag or digest. Jenkins should orchestrate these commands rather than containing application-specific Docker logic in many duplicated shell fragments.

18. Security and production-readiness checklist

18.1 Never bake secrets into images

Never do:

```
ENV OPENAI_API_KEY=secret
COPY .env /app/.env
ARG PRIVATE_TOKEN=secret
```

Deleting the secret in a later layer does not reliably remove it from the earlier layer.

Use:

- BuildKit secret mounts during build
- Compose secrets locally
- Runtime secret manager in deployed environments
- Short-lived workload identity where available

Docker explicitly warns that ARG and ENV are inappropriate for build secrets because their values can persist in the final image or metadata. ([Docker Documentation](#))

18.2 Use minimal trusted base images

Prefer:

- Official language images
- Approved internal base images
- Minimal Debian slim or vetted hardened images
- Digest-pinned references
- Regularly rebuilt patched bases

Avoid:

- Unknown public images
- Unmaintained GenAI “all-in-one” images
- Images containing unnecessary shells, compilers, cloud CLIs, or package managers
- Floating latest tags in controlled environments

Docker recommends minimal trusted base images and digest pinning for stronger reproducibility. ([Docker Documentation](#))

18.3 Vulnerability scanning

Example:

```
docker scout cves genai-rag-api:dev
```

A scanner examines OS packages, language packages, and known vulnerability data. Docker Scout can analyze image contents and report detected packages and vulnerabilities. ([Docker Documentation](#))

A mature policy distinguishes:

- Critical exploitable vulnerability
- Vulnerability with available fix
- Unreachable package
- Development-only dependency
- Accepted exception with expiration
- Base-image issue versus application dependency

Do not merely count CVEs.

18.4 SBOM

An SBOM should identify components such as:

- Package name
- Version
- Licence
- Package identifier
- Dependency source

It supports incident response questions such as:

Which deployed images contain this vulnerable library?

18.5 Provenance

Build provenance records information such as:

- Build process
- Source reference
- Builder
- Build parameters
- Input materials

It helps establish that the deployed artifact came from the expected source and build process.

18.6 Image signing

At a high level:

```
cosign sign registry.example.com/genai-rag-api@sha256:...
```

Verification:

```
cosign verify registry.example.com/genai-rag-api@sha256:...
```

Cosign supports signing container images and storing signatures alongside registry artifacts.
([Sigstore](#))

A deployment policy can reject unsigned or incorrectly signed images.

18.7 Least privilege

Apply:

```
user: non-root
read_only: true

cap_drop:
  - ALL

security_opt:
  - no-new-privileges:true
```

Avoid unless explicitly required:

```
privileged: true
network_mode: host
cap_add:
  - SYS_ADMIN
```

Never mount the Docker socket into an ordinary application container:

```
- /var/run/docker.sock:/var/run/docker.sock
```

That frequently grants effective control over the host Docker daemon.

18.8 Read-only root filesystem

```
read_only: true
tmpfs:
  - /tmp:size=256m
```

Explicitly mount only directories that must be writable:

- /tmp
- Model cache
- Database data
- Generated output directory

Compose supports creating services with a read-only root filesystem. ([Docker Documentation](#))

18.9 Resource limits

```
mem_limit: 2g
cpus: 2.0
pids_limit: 256
```

Benefits:

- Prevents one worker from consuming the entire laptop or host
- Exposes realistic capacity problems locally
- Makes OOM behavior observable
- Protects databases from ingestion spikes
- Prevents uncontrolled process creation

Limits are not performance targets. Measure actual usage and provide headroom.

18.10 Restart policies

Options include:

```
restart: "no"
restart: on-failure
restart: on-failure:3
restart: always
restart: unless-stopped
```

Docker distinguishes restart behavior based on exit status and whether the container was manually stopped. ([Docker Documentation](#))

Recommended local choices:

- API: unless-stopped
- Worker: on-failure:3 or unless-stopped
- PostgreSQL/Qdrant/Redis: unless-stopped
- One-time migration/model warmup: "no"

A restart policy does not fix a bad configuration. It can instead produce a restart loop that obscures the first error.

18.11 Deterministic builds

Use:

- Exact dependency versions
- Dependency hashes
- Approved base-image digest
- Git-SHA metadata
- Controlled package repositories
- Consistent build platform
- Build provenance
- No network-fetched unversioned assets
- No `curl ... | sh`
- No floating model revision

Avoid:

```
RUN pip install fastapi
RUN apt-get upgrade -y
RUN git clone https://example.com/repo
```

Prefer:

```
RUN pip install --require-hashes -r requirements.lock
```

18.12 Reduce image size

Use:

- Multi-stage builds
- `.dockerignore`
- `--no-install-recommends`
- Runtime-only OS libraries
- No compilers in runtime
- No notebooks or local datasets
- No model weights in the general API image
- No pip cache in the final image
- Correct layer ordering

Do not optimize for image size at the expense of debuggability or compatibility without measurement. A slightly larger, well-maintained Debian slim image may be safer than an extremely minimal image that breaks native AI dependencies.

19. Troubleshooting guide

19.1 Container exits immediately

Check all containers:

```
docker ps --all
```

Read logs:

```
docker logs genai-api
```

Inspect exit state:

```
docker inspect \
  --format='exit={{.State.ExitCode}} error={{.State.Error}}
  oom={{.State.OOMKilled}}' \
  genai-api
```

Common causes:

- Main command completed successfully
- Incorrect module path
- Missing environment variable
- Dependency connection failed during startup
- Command not found
- Permission error
- Architecture mismatch
- OOM kill

Test the image with a shell:

```
docker run --rm -it \
  --entrypoint sh \
  genai-rag-api:dev
```

19.2 Port already in use

Error:

```
bind: address already in use
```

Find the process:

```
lsof -nP -iTCP:8000 -sTCP:LISTEN
```

Change only the host port:

```
docker run -p 8001:8000 genai-rag-api:dev
```

Compose:

```
API_PORT=8001 docker compose up -d api
```

The container still listens on 8000.

19.3 Service cannot resolve another service

Incorrect:

```
DATABASE_HOST=localhost
```

Correct:

```
DATABASE_HOST=postgres
```

Check DNS from the API:

```
docker compose exec api \
python -c "import socket; print(socket.gethostbyname('postgres'))"
```

Check connection:

```
docker compose exec api python -c "
import socket
s = socket.create_connection(('postgres', 5432), timeout=3)
print('connected')
s.close()
"
```

Verify networks:

```
docker inspect genai-local-api-1 \
--format='{{json .NetworkSettings.Networks}}'
```

Both services must be attached to the same network.

19.4 Permission errors on mounted volumes

Inspect container identity:

```
docker compose exec api id
```

Inspect mount configuration:

```
docker inspect genai-local-api-1 \
--format='{{json .Mounts}}'
```

Typical causes:

- Host directory owned by another UID
- Named volume initialized by root
- Read-only bind mount used for a writable path
- Model cache owned by root from a previous run

- SELinux labeling issue on supported Linux systems

Repair a named volume carefully:

```
docker run --rm \
  --user root \
  --volume genai-local_hf_cache:/cache \
  alpine:3.21 \
  chown -R 10001:10001 /cache
```

Avoid solving permission errors by running the application permanently as root.

19.5 Health check fails

Inspect health history:

```
docker inspect \
  --format='{{json .State.Health}}' \
  genai-local-api-1
```

Run the exact command inside the container:

```
docker compose exec api python -c "
import urllib.request
print(
    urllib.request.urlopen(
        'http://127.0.0.1:8000/health/ready',
        timeout=2
    ).read()
)
"
```

Check:

- Is the correct port used?
 - Does the health-check utility exist?
 - Is startup time longer than `start_period`?
 - Does readiness block indefinitely?
 - Is the endpoint protected by authentication?
 - Is it checking an optional dependency as mandatory?
 - Is DNS available?
-

19.6 ARM64 versus AMD64 mismatch

Typical error:

```
exec format error
```

Check host:

```
uname -m
```

Check image:

```
docker image inspect \
  --format='{{.Architecture}}/{{.Os}}' \
  genai-rag-api:dev
```

Build explicitly:

```
docker buildx build \
  --platform linux/amd64 \
  --load \
  --tag genai-rag-api:amd64 \
  .
```

Or multi-platform:

```
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  --push \
  --tag "${IMAGE}" \
  .
```

Emulated builds can be slower, especially for native scientific Python packages. Native multi-architecture builders are preferable for CPU-intensive compilation.

19.7 Out-of-memory

Inspect:

```
docker inspect \
  --format='oom={{.State.OOMKilled}} exit={{.State.ExitCode}}' \
  container-name
```

Monitor:

```
docker stats
```

Common GenAI causes:

- Model too large
- Multiple workers each loading the model
- Large embedding batches
- Unbounded document parsing
- Large request concurrency
- Memory leak
- Large in-memory vector collection
- Insufficient /dev/shm
- CPU and GPU copies of the same tensors

Possible corrections:

- Reduce worker count

- Reduce batch size
 - Stream document processing
 - Bound queue concurrency
 - Increase memory limit with measurement
 - Move model inference to a separate service
 - Increase shm_size when shared memory is the actual issue
 - Ensure caches have eviction limits
-

19.8 GPU is not visible

Check the host:

```
nvidia-smi
```

Check Docker GPU access:

```
docker run --rm \
  --gpus all \
  nvidia/cuda:12.9.0-base-ubuntu22.04 \
  nvidia-smi
```

Check application container:

```
docker exec model-container nvidia-smi
```

Check framework:

```
docker exec model-container python -c "
import torch
print('cuda_available=', torch.cuda.is_available())
print('device_count=', torch.cuda.device_count())
"
```

Common causes:

- NVIDIA driver missing
 - Container Toolkit missing or not configured
 - Docker daemon not restarted after toolkit setup
 - GPU not requested with --gpus or Compose reservation
 - Incompatible framework/CUDA build
 - CUDA_VISIBLE_DEVICES hides the GPU
 - Running Docker inside an environment without GPU pass-through
 - GPU memory already exhausted
-

19.9 Qdrant storage problems

For local development, prefer a named volume:

```
- qdrant_data:/qdrant/storage
```


Qdrant warns that some Docker/WSL bind-mounted Windows filesystems may not provide the POSIX filesystem behavior expected by its storage engine. ([Qdrant](#))

Check logs:

```
docker compose logs --tail=200 qdrant
```

Check disk:

```
docker system df
docker volume inspect genai-local_qdrant_data
```

20. Common mistakes

Mistake 1: Copying the complete repository before dependency installation

Bad:

```
COPY . .
RUN pip install -r requirements.txt
```

Every code change invalidates the dependency layer.

Mistake 2: Baking .env into the image

Bad:

```
COPY .env /app/.env
```

The secret may remain in an image layer permanently.

Mistake 3: Using localhost for Compose dependencies

Bad:

```
QDRANT_URL=http://localhost:6333
```

Correct:

```
QDRANT_URL=http://qdrant:6333
```

Mistake 4: Storing PostgreSQL or Qdrant data in the container layer

Removing the container deletes the data.

Mistake 5: Using depends_on without readiness checks

Starting PostgreSQL before the API does not mean PostgreSQL is ready.

Mistake 6: Running API and ingestion in the same process

A document parsing or embedding spike can degrade API latency and complicate scaling and restart behavior.

Mistake 7: Running four workers with a large model

Each worker may load its own model copy, multiplying RAM or VRAM use.

Mistake 8: Using floating tags everywhere

```
python:latest
qdrant/qdrant:latest
genai-rag-api:latest
```

This makes builds and deployments difficult to reproduce.

Mistake 9: Installing debugging tools in the runtime image

Keep normal runtime images small. Use temporary debug containers or purpose-built debug images when deeper diagnostics are required.

Mistake 10: Treating a container as a virtual machine

Do not:

- SSH into it for routine administration
- Manually patch a running container
- Install packages interactively and expect persistence
- Keep important files only in its writable layer

Change the Dockerfile, rebuild, and replace the container.

21. Hands-on exercises

Exercise 1 – Build cache and non-root verification

Goal

Understand layers, cache invalidation, and runtime identity.

Steps

1. Build the image:

```
docker build \
  --progress=plain \
  --tag genai-rag-api:exercise1 \
  .
```

1. Build it again without changes.
2. Observe which steps show cached results.
3. Modify one Python source file.
4. Rebuild and verify that dependency installation remains cached.
5. Modify `requirements.lock`.
6. Rebuild and observe that the dependency layer runs again.
7. Verify the runtime user:

```
docker run --rm \
  --entrypoint id \
  genai-rag-api:exercise1
```

1. Inspect layers:

```
docker history genai-rag-api:exercise1
```

Expected learning

You should be able to explain why dependency metadata is copied before application source.

Exercise 2 – Diagnose a Compose networking failure

Goal

Practise service-name DNS and health-check debugging.

Steps

1. Temporarily change:

```
DATABASE_HOST: postgres
```

to:

```
DATABASE_HOST: localhost
```

1. Start the stack:

```
docker compose up --build -d
```

1. Inspect:

```
docker compose ps
docker compose logs --tail=100 api
```

1. Enter the API container:

```
docker compose exec api sh
```

1. Resolve the correct hostname:

```
python -c "import socket; print(socket.gethostbyname('postgres'))"
```

1. Restore DATABASE_HOST=postgres.

2. Recreate the API:

```
docker compose up -d --force-recreate api
```

Expected learning

You should be able to explain why localhost refers to the current container rather than another Compose service.

Exercise 3 – Create a multi-platform immutable image

Goal

Produce a CI-ready artifact.

Steps

1. Create a Buildx builder.
2. Calculate the Git SHA.
3. Build for AMD64 and ARM64.
4. Export registry cache.
5. Generate SBOM and provenance.
6. Push the Git-SHA-tagged image.
7. Inspect its manifest and digest.
8. Run a vulnerability scan.
9. Record:

```
Git commit
Image tag
Image digest
Supported platforms
SBOM presence
Provenance presence
Scan result
```

Example command:

```
GIT_SHA="$(git rev-parse --short=12 HEAD)"
REPOSITORY="registry.example.com/genai-rag-api"
IMAGE="${REPOSITORY}:${GIT_SHA}"
CACHE="${REPOSITORY}:buildcache"

docker buildx build \
  --platform linux/amd64,linux/arm64 \
  --cache-from type=registry,ref="${CACHE}" \
  --cache-to type=registry,ref="${CACHE}",mode=max \
  --build-arg VCS_REF="$(git rev-parse HEAD)" \
  --build-arg BUILD_DATE="$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
  --sbom=true \
  --provenance=mode=max \
  --tag "${IMAGE}" \
  --push \
  .

docker buildx imagetools inspect "${IMAGE}"
docker scout cves "${IMAGE}"
```

22. Senior-engineer design summary

A production-minded GenAI container design should look like:

```
One application image
├── FastAPI API command
└── Ingestion-worker command

External runtime state
├── PostgreSQL
├── Redis
├── Qdrant
├── Object storage
└── Model cache or model service

Image characteristics
├── Multi-stage
├── Non-root
├── Exec-form process
├── Graceful SIGTERM
├── Health endpoints
├── Read-only root filesystem
├── Minimal runtime dependencies
├── Locked Python dependencies
├── Git SHA metadata
├── SBOM and provenance
└── Signed immutable digest
```

The core principle is:

| Containers package processes, not entire mutable servers.

23. Interview Q&A

1. What is the difference between a Docker image and a container?

An image is a static, layered application package. A container is a runtime instance of that image with a process, network namespace, and writable container layer. Multiple containers can run independently from the same image.

2. Why should model weights usually remain outside a FastAPI application image?

Weights make images extremely large, slow pulls and cold starts, couple model releases to application releases, and complicate licensing and vulnerability workflows. Store them in a model registry, object store, node cache, or mounted volume and pin the exact model revision.

3. Why copy dependency files before application source in a Dockerfile?

Dependencies change less frequently than source code. Copying and installing them first allows Docker to reuse the expensive dependency layer when only application code changes.

4. Why is exec-form ENTRYPOINT or CMD important?

Exec form starts the application directly as PID 1, enabling correct SIGTERM delivery and graceful shutdown. Shell form inserts `/bin/sh -c`, which can interfere with signal forwarding unless carefully handled.

5. What is the difference between liveness and readiness?

Liveness indicates whether the process itself is functioning. Readiness indicates whether it can currently serve normal traffic, including required dependencies or loaded resources. Liveness should generally not fail merely because a remote dependency is temporarily unavailable.

6. Does Compose depends_on guarantee that PostgreSQL is ready?

Not by itself. Basic depends_on controls startup order. Use a PostgreSQL health check and condition: `service_healthy` to wait until it accepts connections. The application should still implement retry logic because dependencies can fail after startup.

7. Why separate the API and ingestion worker?

They have different latency, scaling, retry, shutdown, CPU, and memory characteristics. Separation prevents document parsing and embedding spikes from degrading API traffic and allows independent scaling and failure recovery.

8. Would you run multiple Uvicorn or Gunicorn workers in a GenAI container?

Only after considering memory duplication. Each worker may load its own model, cache, and connection pools. Under Kubernetes, I generally prefer one Uvicorn process per container and scale containers. On a single Compose host, multiple workers may be reasonable for a lightweight API without large in-process models.

9. How does a GPU become visible inside a Docker container?

The host provides a supported NVIDIA driver. NVIDIA Container Toolkit integrates GPU devices and libraries with Docker. The container requests devices using `--gpus` or a Compose GPU reservation and contains a compatible GPU-enabled framework and CUDA user-space stack.

10. What causes an ARM64/AMD64 exec format error?

The image or executable was built for a different CPU architecture than the runtime host. Build an image for the target platform or publish a multi-platform manifest using Buildx.

11. What is the difference between a Git-SHA tag and an image digest?

A Git-SHA tag provides traceability to source code, but a registry tag can technically be moved. A digest identifies exact image content and is immutable. Production promotion should ultimately track the digest.

12. What does “build once and promote” mean?

Build and verify one image, then deploy that exact digest to development, staging, and production. Environment-specific configuration and secrets are supplied at runtime. Rebuilding for each environment risks producing different artifacts.

13. How do you prevent secrets from entering image layers?

Exclude secret files with `.dockerignore`, never use secret values in Dockerfile ARG or ENV, use BuildKit secret mounts for build-time credentials, and inject runtime secrets through the deployment platform or file-mounted Compose secrets.

14. What do SBOM, provenance, and signing provide?

An SBOM describes what software is inside the image. Provenance describes how and from which inputs it was built. Signing establishes an identity-backed statement about the artifact. Together they support vulnerability response, traceability, and deployment-policy enforcement.

15. Why use a read-only root filesystem?

It prevents normal runtime processes from modifying the application image filesystem, limits persistence opportunities after compromise, and identifies hidden filesystem dependencies. Explicit writable volumes or `tmpfs` mounts are provided only where required.

16. A model worker exits with code 137. What do you investigate?

I check `.State.OOMKilled`, container memory limits, host memory pressure, number of model copies, embedding batch size, request concurrency, shared-memory use, and GPU memory. Code 137 means SIGKILL; OOM is common but not the only possible cause.